AC.2.1. Diseña y construye tu web

Modalidad: individual.
Tecnologías: HTML y CSS.
Evaluación: tres calificaciones independientes: desarrollo de interfaces (RA2), accesibilidad (RA5) y usabilidad (RA6).

1. El reto

Diseña y desarrolla una web de temática libre que permita descubrir un catálogo, consultar sus elementos y preparar una compra, reserva o solicitud.

Puedes crear una tienda, una web de entradas para eventos, un servicio de alquiler, una biblioteca con reservas, un catálogo de actividades u otra propuesta equivalente.

El proyecto se realizará en tres fases:

1. Definición y diseño.
2. Desarrollo con HTML y CSS.
3. Comprobación y mejora.

Esta será la primera versión de una web que ampliarás en próximas prácticas. Por ello, debes cuidar tanto el resultado como la organización del código. Trabajarás de forma profesional, subiendo tu código a GitHub y publicando tu web en GitHub Pages.

2. Define tu propuesta antes de programar

Crea un documento breve en la carpeta docs del repositorio. Incluye:

Aspecto	Qué debes concretar
Nombre y propósito	Qué ofrece tu web y qué necesidad resuelve.
Usuarios	A quién se dirige, qué conocimientos tienen y qué necesidades pueden afectar al diseño. Evita definirlos simplemente como «todo el mundo».
Contexto de uso	Desde dónde y en qué circunstancias la utilizarán: dispositivo, tiempo disponible, posibles distracciones, etc.

Tareas principales	Tres tareas concretas que podrán realizar los usuarios.
Recorrido principal	Qué pasos seguirá una persona desde que entra hasta que prepara una compra, reserva o solicitud.
Información necesaria	Qué necesita conocer el usuario en cada paso para poder decidir y continuar.

Extensión orientativa: una o dos páginas. Se valorará que las decisiones sean concretas y coherentes, no la longitud del documento.

3. Diseña las páginas y revisa los bocetos

Prepara un **boceto de cada una de las seis páginas** antes de desarrollarlas. Puedes hacerlo en papel y fotografiarlo, o utilizar una herramienta digital.

Los bocetos deben mostrar:

- Navegación y ubicación de los elementos principales.
- Encabezados, información y acciones de cada página.
- Campos y agrupaciones de los formularios.
- Relación entre las páginas.

No hace falta definir todavía todos los colores ni reproducir el diseño final con precisión.

Revisión con un compañero

Entrega los bocetos a un compañero y pídele que intente resolver estas tres tareas:

1. Encontrar un elemento del catálogo y consultar sus características.
2. Localizar la selección o carrito e identificar cómo continuar.
3. Localizar el formulario y comprender qué información debe introducir.

Durante la revisión, evita indicarle dónde debe pulsar o qué camino seguir.

Registra brevemente:

Tarea	Qué entendió o intentó hacer	Dificultad detectada	Cambio propuesto

Anota quién ha revisado los bocetos y la fecha. Explica qué sugerencias incorporas y cuáles descartas, justificando la decisión.

Guarda los bocetos iniciales y la revisión en el repositorio antes de empezar el desarrollo.

4. Páginas obligatorias

La web tendrá, como mínimo, **seis páginas HTML conectadas entre sí**

A. Página inicial

Debe permitir comprender rápidamente:

- Qué ofrece la web.
- A quién va dirigida.
- Cómo acceder al catálogo.
- Cuál es la acción principal que puede realizar el visitante.

Incluye contenido representativo de tu temática y una selección de elementos destacados.

B. Página de acceso

Incluye:

- Campo de correo electrónico o nombre de usuario.
- Campo de contraseña.
- Etiquetas visibles y asociadas a los controles.
- Botón de acceso.
- Una indicación clara de que se trata de una demostración sin autenticación real.

No es necesario crear cuentas ni comprobar credenciales. No solicites datos reales para probarla.

C. Página de formulario

Elige un formulario que tenga sentido para tu proyecto: inscripción, solicitud de reserva, personalización de un pedido, alta como socio, etc.

Debe incluir **al menos ocho campos de información**, utilizando:

- Un campo de texto.
- Un campo de correo electrónico.
- Un desplegable select.

- Un grupo de botones de opción.
- Una casilla de verificación con un propósito comprensible.
- Un área de texto textarea.
- Otros dos campos adecuados a la temática: fecha, número, teléfono, texto, etc.

Un grupo de botones de opción cuenta como **un campo**, no como varios. Los botones de envío tampoco cuentan.

Además:

- Agrupa la información relacionada mediante fieldset y legend.
- Asocia cada etiqueta con su control.
- Distingue los campos obligatorios de los opcionales.
- Utiliza los tipos de campo y las restricciones HTML apropiados: required, min, max, etc., cuando proceda.
- Incluye instrucciones cuando el formato esperado pueda generar dudas.
- Utiliza un texto de envío específico, como «Solicitar reserva», en lugar de uno ambiguo.

No añadas campos sin utilidad solo para alcanzar el mínimo. Si tu temática no los justifica, revisa el propósito del formulario.

D. Página de catálogo

Incluye **al menos seis elementos** con contenido diferente y coherente.

Cada ficha mostrará:

- Nombre.
- Imagen.
- Descripción breve.
- Precio u otro dato relevante: disponibilidad, duración, categoría, etc.

El catálogo se organizará con **CSS Grid**.

Al menos una ficha tendrá un enlace a su página de detalle. No es necesario desarrollar seis páginas de detalle: identifica claramente cuál está disponible y evita enlaces que lleven a información de otro elemento.

Además, diseña los siguientes componentes:

Componente	Requisitos
Buscador	Campo de búsqueda con etiqueta visible y botón «Buscar».
Filtros	Al menos dos criterios adecuados a la temática, con controles apropiados y un botón «Aplicar filtros».
Resumen de resultados	Un texto de ejemplo, como «Mostrando 1–6 de 18 resultados».
Paginación	Controles «Anterior», números de página y «Siguiente», con la página actual claramente identificada.

En esta entrega se evalúa su estructura y presentación, no su funcionamiento dinámico. No es necesario buscar, filtrar, cambiar de página ni crear otros archivos de catálogo.

Los campos deben permitir escribir y seleccionar opciones. Utiliza botones de tipo button para las acciones simuladas, evitando envíos o recargas involuntarias. No utilices enlaces con destinos inexistentes.

Incluye un aviso visible junto a estas herramientas:

«Prototipo: la búsqueda, los filtros y la paginación todavía no modifican los resultados».

Cuida especialmente:

- Las etiquetas asociadas a los campos.
- La agrupación y organización de los filtros.
- El acceso mediante teclado y el foco visible.
- La diferenciación de la página actual mediante algo más que el color.
- La coherencia de estos componentes con el resto de la web.

E. Página de detalle

Desarrolla el detalle de uno de los elementos del catálogo. Incluye:

- Nombre e imagen.
- Descripción ampliada.
- Características relevantes.
- Precio o condiciones de la reserva o solicitud.
- Una acción principal claramente identificada.
- Una forma de volver al catálogo.

La información debe permitir decidir si ese elemento interesa al usuario.

F. Página de carrito o selección

Representa una selección de ejemplo con **al menos dos elementos**.

Incluye:

- Identificación de los elementos seleccionados.
- Cantidades, cuando sean pertinentes.
- Precio unitario, subtotales y total, si tu proyecto utiliza precios.
- Controles previstos para modificar cantidades y eliminar elementos.
- Acceso para seguir explorando el catálogo.
- Acceso al siguiente paso, como completar la solicitud.

Los datos y los cálculos pueden estar escritos directamente en el HTML. Si tu temática utiliza reservas en lugar de compras, adapta los términos y el resumen.

5. Qué debe funcionar en esta entrega

Debe funcionar:

- La navegación entre páginas.
- Los enlaces internos.
- La interacción con los controles HTML: escribir, seleccionar y marcar opciones.
- La validación nativa del formulario.
- La navegación mediante teclado.

Se representará como prototipo:

- El acceso de usuarios.
- La incorporación y eliminación de elementos del carrito.
- El recálculo de importes.
- El envío y almacenamiento de datos.
- La recuperación del catálogo desde un servicio.

Identifica estas limitaciones con un aviso visible y explícalas en el README. No muestres mensajes que hagan creer que se ha realizado una compra o enviado una solicitud real.

No se exige JavaScript, PHP ni una base de datos.

6. Requisitos de HTML y CSS — RA2

Estructura HTML

- Utiliza una estructura semántica adecuada: header, nav, main, footer y, donde corresponda, section y article.
- Mantén una jerarquía comprensible de encabezados.
- Emplea enlaces para navegar y botones para acciones.
- Utiliza listas, imágenes y formularios con las etiquetas apropiadas.
- Conserva los div que sean agrupaciones genéricas: no es necesario convertirlos todos en secciones.

Organización de los estilos

- Utiliza hojas CSS externas y comparte los estilos comunes entre páginas.
- Define una apariencia coherente para tipografía, colores, enlaces, botones y formularios.
- Reutiliza clases para componentes repetidos y crea alguna variante, como una tarjeta destacada o un botón secundario.
- Utiliza selectores de tipo, clase, algún selector contextual o de atributo y estados como :hover y :focus-visible.
- Debes poder explicar un ejemplo de herencia y otro de resolución de estilos mediante la cascada.
- Evita solucionar conflictos acumulando reglas repetidas o usando !important sin justificación.

Modelo de caja

- Controla conscientemente márgenes, rellenos, bordes y dimensiones.
- Utiliza box-sizing de forma coherente.
- Evita alturas fijas que recorten textos y espacios creados con párrafos vacíos o saltos de línea repetidos.

Flexbox

Utiliza Flexbox en **al menos dos componentes diferentes**, por ejemplo:

- Cabecera o navegación.
- Grupo de acciones.

- Contenido interior de las tarjetas.
- Fila de un elemento del carrito.

Demuestra un uso adecuado de dirección, separación y alineación. En algún componente, utiliza flex-wrap o controla el dimensionamiento de sus elementos mediante flex.

Grid

- Organiza el catálogo en una cuadrícula de varias columnas.
- Utiliza columnas flexibles y separación mediante gap.
- Permite que las filas crezcan cuando el contenido lo necesite.
- Aplica en alguna zona una distribución mediante áreas o un elemento que abarque varias filas o columnas, con una finalidad visual razonable.

No alteres el orden visual de manera que contradiga el orden de lectura y navegación.

Comportamiento al cambiar el tamaño de la ventana

En esta primera entrega no se exige un diseño específico para el móvil. La adaptación completa a diferentes dispositivos se desarrollará en la segunda práctica.

Comprueba la web con anchuras del área de visualización de **1024 y 1440 píxeles**, utilizando el modo de diseño adaptable de DevTools.

En ambas anchuras:

- Los contenidos y controles deben permanecer visibles y utilizables.
- No debe haber superposiciones, recortes ni desplazamiento horizontal general de la página.
- Los componentes deben aprovechar correctamente el espacio mediante las técnicas de Flexbox, Grid y dimensionamiento trabajadas.
- Las cajas deben admitir el crecimiento de sus textos.

No es obligatorio cambiar el número de columnas ni reorganizar la navegación entre esas dos anchuras.

Además, realiza por separado la comprobación de ampliación al 200 % indicada en el apartado de accesibilidad. En esta prueba se comprobará que no se pierde contenido ni funcionalidad; no se exige todavía una distribución específica para móvil.

DevTools

Documenta **tres comprobaciones breves**, con capturas y explicación:

1. Una propiedad heredada o una regla sobrescrita y el valor que finalmente se aplica.
2. El modelo de caja de un componente.
3. La distribución de un contenedor Flexbox o Grid mediante el inspector.

Las capturas deben mostrar elementos de tu proyecto. Explica qué has comprobado o corregido.

7. Requisitos de usabilidad — RA6

El diseño final debe reflejar el análisis inicial:

- El propósito de la web se entiende sin explicación del autor.
- La navegación mantiene nombres y posiciones coherentes.
- Se reconoce la página o apartado actual.
- La información importante destaca por su jerarquía y organización.
- Los elementos relacionados aparecen agrupados.
- Los enlaces y botones indican qué ocurrirá al utilizarlos.
- Los formularios solicitan información pertinente y ofrecen instrucciones suficientes.
- Se puede volver atrás o continuar sin quedar en una página sin salida.
- Las limitaciones del prototipo se comunican de forma comprensible.

Prueba final con un compañero

Repite las tres tareas de la revisión inicial sobre la web desarrollada. El compañero debe intentarlas sin recibir instrucciones sobre dónde pulsar.

Registra:

- Si completa cada tarea de forma autónoma, con ayuda o no la completa.
- Dificultades observadas.
- Cambios realizados después de la prueba.

Distingue los problemas de uso de las funciones que todavía no están implementadas. Valora al menos dos observaciones concretas, aunque alguna no requiera modificar el diseño.

8. Requisitos de accesibilidad — RA5

Comprueba todas las páginas, prestando especial atención a:

Aspecto	Comprobación
Estructura	Idioma del documento, título de página descriptivo y encabezados organizados.
Imágenes	Alternativas textuales adecuadas. Las imágenes puramente decorativas tendrán alt="".
Formularios	Etiquetas visibles y asociadas; agrupaciones comprensibles e instrucciones para los campos. El placeholder no sustituye a la etiqueta.
Teclado	Se llega a los enlaces y controles, se pueden utilizar y no aparecen trampas de teclado.
Foco	Se ve claramente qué elemento tiene el foco y el recorrido resulta lógico.
Contraste	Texto normal con contraste mínimo de 4,5:1; texto grande, 3:1. Como referencia, texto grande es al menos 24 px, o aproximadamente 18,7 px en negrita.
Uso del color	Los estados y la información importante no se comunican exclusivamente mediante el color.
Ampliación	Al ampliar al 200 %, el contenido y los controles siguen disponibles, sin textos recortados o superpuestos que impidan utilizarlos.

Entrega una tabla de comprobación con estas columnas:

Aspecto · Página o elemento revisado · Método utilizado · Resultado · Corrección realizada o problema pendiente.

Incluye evidencias de teclado/foco, contraste y ampliación. En contraste, indica los colores comprobados y la relación obtenida.

Esta revisión es parcial: **no permite afirmar que toda la web cumple WCAG A, AA o AAA**.

9. Entrega y publicación en GitHub Pages

El proyecto se entregará mediante un **repositorio individual de GitHub** y se publicará en **GitHub Pages**, para que pueda consultarse directamente desde un navegador.

Paso 1. Prepara el repositorio

Crea un repositorio público con un nombre relacionado con tu proyecto. Si ya tienes uno para esta práctica, continúa trabajando en él.

Organiza los archivos siguiendo una estructura similar a esta:

```
index.html
login.html
formulario.html
catalogo.html
detalle.html
carrito.html
css/
  estilos.css
img/
docs/
  ...
README.md
```

- La página inicial debe llamarse exactamente index.html y estar en la raíz del repositorio.
- Guarda en docs el análisis, los bocetos, las revisiones y las comprobaciones.
- Realiza commits durante el desarrollo, con mensajes que expliquen los cambios.
- Utiliza datos ficticios. El repositorio y la web serán públicos: no incluyas contraseñas reales, claves de servicios ni información personal de compañeros.

Paso 2. Crea tu copia local y sube los primeros archivos

Trabajarás con dos copias del proyecto:

- **Repositorio local:** la carpeta de tu ordenador donde editas y guardas el trabajo.
- **Repositorio remoto:** la copia alojada en GitHub, que utilizarás para entregar y publicar.

Guardar un archivo no lo sube automáticamente a GitHub. Para hacerlo, tendrás que registrar los cambios con un **commit** y enviarlos con **push**.

A. Comprueba que tienes Git

Abre una terminal y ejecuta:

git --version

Si muestra un número de versión, Git está instalado. Si no, deberás instalarlo.

Si todavía no has configurado tu identidad para los commits, ejecuta estos comandos sustituyendo los datos de ejemplo:

git config --global user.name "Tu nombre"

git config --global user.email "Tu correo"

Utiliza el correo asociado a tu cuenta de GitHub o la dirección privada noreply que proporciona GitHub. Esta identidad se incorpora a los commits; **no sirve para iniciar sesión**.

B. Clona el repositorio

1. Crea el repositorio en GitHub y marca la opción de añadir un **README**, para que tenga un primer contenido.
2. En el repositorio, pulsa **Code → HTTPS** y copia la dirección.
3. Abre una terminal en la carpeta donde guardas tus proyectos. No lo hagas dentro de otro repositorio.
4. Ejecuta lo siguiente, sustituyendo la dirección por la tuya:

git clone https://github.com/TU-USUARIO/TU-REPOSITORIO.git

Git creará una carpeta con el nombre del repositorio. Entra en ella:

cd TU-REPOSITORIO

Esta es la carpeta que debes abrir en tu editor y donde debes trabajar. No necesitas ejecutar git init.

Si ya habías empezado la web en otra carpeta, copia sus archivos dentro de esta carpeta clonada. No copies carpetas .git de otros proyectos.

C. Registra tus primeros cambios

Crea o copia los archivos del proyecto y guárdalos. Desde la terminal, situada dentro del repositorio, ejecuta:

git status

Este comando muestra los archivos nuevos o modificados. Revisa que solo aparezcan archivos que quieras incluir en el proyecto.

Después ejecuta:

git add .

git commit -m "Añade estructura inicial de la web"

- git add . prepara los cambios de la carpeta actual y sus subcarpetas.
- git commit registra una versión en el repositorio **local**.

El commit todavía no ha enviado nada a GitHub.

D. Envía los cambios a GitHub

Ejecuta:

git push

Si se solicita autenticación, sigue las indicaciones del equipo para iniciar sesión con tu cuenta de GitHub.

Abre el repositorio en GitHub y actualiza la página. Comprueba que aparecen tus archivos y el mensaje del commit.

Paso 3. Repite este ciclo durante el desarrollo

Cada vez que completes una parte coherente del trabajo:

1. Guarda los archivos en el editor.
2. Revisa los cambios.
3. Registra una versión.
4. Envíala a GitHub.

git status

git add .

git commit -m "Describe el cambio realizado"

git push

Ejemplos de mensajes adecuados:

- Añade bocetos y revisión del compañero
- Maqueta el catálogo con Grid
- Corrige etiquetas y foco del formulario

No es necesario hacer un commit por cada línea modificada. Agrupa cambios relacionados y utiliza mensajes que permitan entender la evolución del proyecto.

Una vez configurado GitHub Pages, los cambios enviados a la rama de publicación también actualizarán la web tras completarse el despliegue.

Si trabajas en otro ordenador

La primera vez, clona el repositorio en ese equipo. Si ya lo tienes clonado, antes de empezar y con los cambios locales anteriores guardados y enviados, ejecuta:

git pull

Así incorporarás los cambios que hayas subido desde el otro ordenador.

Antes de cambiar de equipo, termina siempre con **commit y push**. Si Git informa de un conflicto o rechaza una operación, pide ayuda y no utilices push --force para intentar resolverlo.

Paso 3. Comprueba las rutas

Utiliza rutas relativas para enlazar páginas, imágenes y hojas de estilos. Por ejemplo, desde los HTML situados en la raíz:

<link rel="stylesheet" href="css/estilos.css">

<img src="img/producto.jpg" alt="Descripción del producto">

Ver catálogo

Evita rutas de tu ordenador, como C:\Users\..., y rutas que comiencen por /, porque estas últimas apuntan a la raíz del dominio, no necesariamente a la carpeta de tu proyecto.

Los nombres deben coincidir exactamente, incluidas las mayúsculas: Foto.jpg y foto.jpg son nombres diferentes.

Recuerda: las rutas de imágenes utilizadas mediante url(...) en un CSS se calculan desde la ubicación de ese archivo CSS.

Paso 4. Publica la web

Una vez subidos los archivos a GitHub:

1. Abre el repositorio y entra en **Settings**.
2. Selecciona **Pages** en el menú lateral.
3. En **Build and deployment**, selecciona como origen **Deploy from a branch**.
4. Elige la rama **main** —o la rama que estés utilizando— y la carpeta **/(root)**.
5. Pulsa **Save**.
6. Espera a que termine la publicación. Puede tardar varios minutos.
7. Vuelve a **Settings → Pages** y abre **Visit site**.

Selecciona **/(root)** porque allí está index.html. La carpeta docs de esta práctica contiene documentación, no la web.

Estos pasos siguen la guía oficial de configuración de GitHub Pages.

Paso 5. Comprueba la versión publicada

Realiza estas comprobaciones sobre la dirección pública, no solo sobre los archivos de tu ordenador:

- La página inicial se abre correctamente.
- Puedes navegar por las seis páginas.
- Se cargan las imágenes y los estilos.
- Funcionan los enlaces internos y los controles de formulario.
- Puedes recorrer enlaces y controles con el teclado y ver el foco.
- No aparecen páginas inexistentes ni enlaces a archivos locales.

Si detectas un problema, corrígelo en tus archivos, realiza un commit y sube los cambios. GitHub Pages actualizará la web después de un nuevo despliegue.

El login, el envío de formularios y la gestión del carrito siguen siendo **simulaciones**, tal como establece esta práctica. Publicar la web no añade esas funcionalidades.

Paso 6. Completa el README

Incluye:

1. Nombre y descripción del proyecto.
2. **Enlace a la web publicada.**
3. Páginas disponibles y recorrido recomendado.
4. Instrucciones para abrir el proyecto localmente.
5. Funciones representadas pero no implementadas.
6. Dónde se utilizan Flexbox y Grid.
7. Enlaces a la documentación de diseño y comprobación.
8. Procedencia de imágenes y otros recursos.
9. Herramientas de IA utilizadas, si las hubiera, y para qué.

Paso 7. Entrega tres enlaces

En el aula virtual entrega:

Enlace	Para qué se utilizará
Repositorio de GitHub	Revisar el código, la documentación y la evolución del trabajo.
Web publicada en GitHub Pages	Probar la interfaz y su navegación.
Commit de entrega	Identificar la versión exacta que se evaluará.

Para obtener el enlace al commit, abre el historial de commits del repositorio, selecciona el correspondiente a tu entrega y copia su dirección.

Comprueba que la web publicada corresponde a ese commit antes de entregar.

La dirección de GitHub Pages mostrará las actualizaciones posteriores; no queda congelada al entregar. El enlace al commit permitirá recuperar la versión evaluada.

Continúa utilizando este mismo repositorio para la segunda práctica.

10. Comprobación individual

Durante la revisión en clase deberás:

- Localizar y explicar tu código.
- Utilizar DevTools para justificar un comportamiento.
- Realizar una pequeña modificación indicada por mí.

Esta comprobación se realizará sin asistentes de IA. Durante el desarrollo puedes consultar documentación y utilizar las ayudas permitidas, pero debes comprender y poder modificar lo que entregas. Si veo que no eres capaz de explicar tu propio código ni hacer modificaciones similares a lo desarrollado, asumiré que no lo has hecho tú y daré por suspensa la práctica.

11. Evaluación

Se obtendrán **tres notas independientes sobre 10**.

RA2 — Desarrollo de interfaces

Criterio	Puntos
Seis páginas, estructura HTML, formularios y navegación	2,5
Organización del CSS, selectores, herencia y cascada	2,5
Modelo de caja y control de dimensiones y espacios	1
Aplicación de Flexbox	1,5
Aplicación de Grid	1,5
Uso de DevTools	1
Total	**10**

RA6 — Usabilidad

Criterio	Puntos
Propósito, usuarios, contexto, tareas y recorrido	2
Bocetos y revisión previa con un compañero	2
Organización, navegación, claridad y coherencia de la web final	5
Mejoras justificadas	1
Total	**10**

RA5 — Accesibilidad

Criterio	Puntos
Estructura, encabezados y alternativas de las imágenes	2
Etiquetas, agrupaciones e instrucciones de formularios	2
Uso con teclado, orden de navegación y foco visible	2
Contraste, información no dependiente del color y ampliación	2
Comprobaciones documentadas y correcciones	2
Total	**10**

En cada apartado se valorará el grado de cumplimiento: completo y correcto, mayormente correcto, parcial o ausente. La documentación debe corresponderse con la web entregada; declarar que algo se ha comprobado no sustituye a su funcionamiento.